

Belady-Guided Cache Replacement with Uncertainty-Aware Machine Learning

Kareem Alqutob

Department of Electrical and Computer Engineering
Birzeit University, Palestine
Email: 1211756@student.birzeit.edu

Hussain Abughosh

Department of Electrical and Computer Engineering
Birzeit University, Palestine
Email: 1210338@student.birzeit.edu

Abstract—Cache replacement policies strongly influence processor performance by determining which cache blocks are evicted on a miss. Traditional heuristics such as Least Recently Used (LRU) often perform poorly under irregular memory access patterns. Although Belady’s optimal replacement algorithm achieves the minimum possible miss rate, it cannot be applied online due to its dependence on future access information.

This paper presents a Belady guided cache replacement policy that employs lightweight supervised machine learning to approximate optimal eviction decisions while maintaining robust online behavior. An offline phase uses Belady’s algorithm as an oracle to label cache blocks, enabling the training of a simple classifier based on past access features. During execution, an uncertainty-aware confidence mechanism selectively applies machine learning predictions and falls back to LRU when confidence is low. Experimental evaluation using a trace-driven cache simulator shows that the proposed approach improves cache performance over LRU while preserving stability across diverse workloads.

Index Terms—Cache replacement, Belady’s algorithm, machine learning, uncertainty-aware systems, computer architecture

I. INTRODUCTION

Modern processors rely heavily on cache memories to mitigate the performance gap between fast CPU cores and relatively slow main memory [2]. Since cache capacity is limited, cache management policies play a critical role in determining system performance. Among these policies, cache replacement directly influences the cache miss rate by deciding which blocks are evicted when new data must be brought into the cache. Even small improvements in replacement decisions can translate into significant performance gains.

Traditional cache replacement policies such as Least Recently Used (LRU) and Least Frequently Used (LFU) rely on simple heuristics that exploit temporal or frequency locality [2]. While effective for many workloads, these heuristics often struggle under modern application behaviors characterized by irregular memory access patterns, phase changes, and complex reuse behavior [9]. As a result, fixed heuristic-based policies frequently fail to make optimal eviction decisions across diverse workloads.

Belady’s optimal replacement algorithm provides a theoretical upper bound on cache performance by evicting the cache block whose next use occurs farthest in the future [1]. Although this policy achieves the minimum possible miss rate, it is not implementable in practice due to its reliance on future

access information. Nevertheless, Belady’s algorithm serves as an important oracle for guiding the design of practical replacement policies. Recent research has demonstrated that machine learning techniques can approximate Belady-like behavior by learning reuse patterns from past memory accesses [4]–[6].

However, many machine learning-based cache replacement approaches introduce significant complexity or rely on aggressive learning mechanisms that may lead to unstable behavior when predictions are inaccurate [3], [7]. Blindly trusting machine learning models can result in harmful eviction decisions, particularly for workloads that differ from training data or exhibit highly dynamic access patterns.

To address these challenges, this paper proposes a Belady-guided cache replacement policy augmented with an uncertainty-aware machine learning mechanism. A lightweight supervised model is trained offline using Belady’s decisions as an oracle to classify cache blocks based on simple access history features. During execution, machine learning predictions are applied only when sufficient confidence is available; otherwise, the policy safely falls back to LRU. This hybrid approach combines the adaptiveness of machine learning with the robustness of traditional heuristics. Experimental evaluation using a trace-driven cache simulator shows that the proposed policy improves cache performance over LRU while maintaining stable behavior across diverse memory access patterns.

The main contributions of this paper are summarized as follows:

- We propose a Belady-guided cache replacement policy that leverages lightweight supervised machine learning to approximate optimal eviction decisions using only past access information available at runtime.
- We introduce an uncertainty-aware confidence mechanism based on prediction score dispersion that selectively applies learning-based eviction decisions and safely falls back to LRU, guaranteeing bounded worst-case behavior equivalent to the baseline heuristic.
- We design a simple and practical replacement policy that avoids complex deep learning models, enabling low-overhead integration suitable for hardware-oriented implementations.
- We implement the proposed policy in a trace-driven cache simulator and demonstrate through experimental

evaluation that it improves cache hit rate over LRU across diverse workloads while preserving stability in LRU-friendly scenarios.

II. BACKGROUND

A. Cache Memory and Replacement Policies

Cache memories are small, fast storage structures that reduce the average memory access latency by exploiting temporal and spatial locality in program execution [2]. Due to their limited capacity, cache blocks must be evicted when new data is brought into a full cache. The cache replacement policy determines which block is selected for eviction and therefore has a direct impact on cache performance metrics such as miss rate and average memory access time.

Traditional cache replacement policies rely on heuristic assumptions about program behavior. Least Recently Used (LRU) evicts the block that has not been accessed for the longest time, assuming that recently accessed blocks are more likely to be reused. Least Frequently Used (LFU) prioritizes blocks based on access frequency, favoring blocks that have been accessed more often in the past [2]. While these policies are simple and effective for many workloads, they often fail to adapt to irregular access patterns, phase changes, and complex reuse behavior commonly observed in modern applications [9].

B. Belady's Optimal Replacement Algorithm

Belady's optimal replacement algorithm, also known as MIN, provides a theoretical lower bound on cache miss rate [1]. The algorithm evicts the cache block whose next access occurs farthest in the future, or that is never accessed again. By making eviction decisions based on complete knowledge of future memory accesses, Belady's algorithm achieves the minimum possible miss rate for a given cache configuration.

Despite its optimality, Belady's algorithm is not implementable in practical systems, as future memory accesses are unknown at runtime. Nevertheless, Belady's algorithm serves as an important reference and evaluation baseline in cache research. Many modern cache replacement policies are inspired by Belady's behavior and aim to approximate its decisions using information available at runtime [6].

C. Machine Learning for Cache Replacement

Recent advances in machine learning have motivated its application to cache replacement policy design [12]. Machine learning models can learn complex access patterns and reuse behavior from historical data, enabling more informed eviction decisions than fixed heuristics. Supervised learning approaches often rely on offline training using labeled data derived from optimal or near-optimal policies, while reinforcement learning methods adapt policies online based on observed rewards [3], [7].

Although machine learning-based cache replacement has demonstrated promising results, many approaches introduce significant computational overhead or rely on aggressive learning mechanisms that may lead to unstable behavior when

predictions are inaccurate. Lightweight learning models and hybrid designs that combine machine learning with traditional heuristics have therefore emerged as practical alternatives, balancing adaptiveness and robustness [8]. These observations motivate the development of cache replacement policies that leverage machine learning selectively while maintaining safe fallback mechanisms.

D. Prediction Confidence in Learning-Based Policies

In learning-based cache replacement, eviction decisions are inherently risky, as incorrect predictions may result in long-term performance degradation. Prediction confidence provides an indication of how strongly a model favors one decision over others and can be used to assess the reliability of learned policies. Incorporating confidence awareness into decision-making processes has been explored in other domains as a means of improving robustness [12], motivating its application to cache replacement.

III. RELATED WORK

A. Heuristic-Based Cache Replacement Policies

Early cache replacement policies relied on simple heuristics to exploit locality in memory access patterns. Least Recently Used (LRU) and Least Frequently Used (LFU) are among the most widely used policies due to their simplicity and low implementation overhead. However, these heuristics assume stable temporal or frequency locality and often perform poorly under irregular access patterns, streaming workloads, and phase changes, motivating the development of more adaptive replacement strategies.

B. Belady-Inspired Replacement Policies

Belady's optimal replacement algorithm serves as a theoretical benchmark for cache performance by evicting the block whose next access occurs farthest in the future. Although unimplementable in practice, Belady's algorithm has inspired several practical replacement policies that attempt to approximate its behavior using historical access information. Hawkeye [6] is a seminal Belady-guided policy that trains a classifier offline using Belady's decisions to identify cache-friendly and cache-unfriendly blocks. This approach demonstrates that learning from optimal behavior can significantly outperform traditional heuristics while remaining feasible for hardware implementation.

C. Supervised and Imitation Learning Approaches

Supervised learning has been applied to cache replacement by framing eviction decisions as a classification problem. PARROT [5] adopts an imitation learning approach that directly trains a supervised model on Belady's eviction decisions, allowing the policy to closely approximate optimal behavior. These approaches highlight the effectiveness of learning from oracle policies but require careful model design to balance accuracy and runtime overhead.

Deep learning techniques have also been explored. Glider [4] employs a neural network trained offline to predict

reuse distance, enabling Belady-like eviction decisions. While effective, deep learning-based approaches introduce increased complexity and computational cost, limiting their practicality in many systems.

D. Reinforcement Learning-Based Cache Replacement

Reinforcement learning (RL) has been proposed as a means to adapt cache replacement decisions dynamically. Souza and Freitas [3] present an RL-based replacement policy for multicore processors that incorporates coherence and sharing information. CHROME [7] extends this concept by jointly managing cache replacement, bypassing, and prefetching using online reinforcement learning. Although RL-based approaches achieve promising results, they often incur high runtime overhead and may suffer from instability due to continuous online learning.

E. Lightweight Adaptive and Context-Aware Policies

To reduce complexity, lightweight adaptive replacement policies have been proposed. LeCaR [8] dynamically switches between LRU and LFU using online regret minimization, demonstrating that combining simple heuristics with learning mechanisms can yield performance improvements with minimal overhead.

Other works emphasize the importance of contextual information in replacement decisions. Qureshi et al. [9] show that memory-level parallelism (MLP) affects the cost of cache misses and should influence eviction decisions. PACMan [10] focuses on prefetch-aware cache management, highlighting the interaction between replacement and prefetching behavior.

F. Surveys and Reviews

Several survey and review papers provide a broader perspective on machine learning for cache replacement and computer architecture. Pratheeksha and Revathi [11] review machine learning-based cache replacement policies, identifying key challenges and research directions. Wu and Xie [12] present a comprehensive survey of machine learning applications in computer architecture, highlighting cache replacement as a domain where learning-based techniques can outperform traditional heuristics.

G. Summary and Motivation

While prior work demonstrates the potential of machine learning to improve cache replacement, many existing approaches rely on complex models or aggressively trust learning predictions, which can lead to unstable behavior when predictions are inaccurate. These limitations motivate the exploration of lightweight, Belady-guided cache replacement policies that selectively apply machine learning while maintaining safe fallback mechanisms. The proposed work builds upon these insights by introducing an uncertainty-aware hybrid approach that balances adaptiveness and robustness.

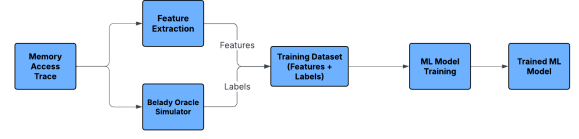


Fig. 1. Offline training phase of the proposed Belady-guided cache replacement policy.

IV. PROPOSED SOLUTION

A. Overview

The proposed solution consists of two phases: an offline training phase and an online cache replacement phase. In the offline phase, Belady’s optimal replacement algorithm is used as an oracle to label cache blocks based on their future reuse behavior. These labels are then used to train a lightweight machine learning model using simple access history features. During the online phase, the trained model guides eviction decisions at runtime without access to future information. To avoid harmful mispredictions, an uncertainty-aware mechanism selectively applies machine learning decisions and falls back to LRU when prediction confidence is low.

B. Offline Training Phase

The offline training phase operates on complete memory access traces and is not part of the runtime cache system. Given a trace and a cache configuration, Belady’s optimal replacement algorithm is simulated to determine optimal eviction decisions. At each eviction event, cache blocks are labeled as cache-friendly or cache-unfriendly depending on whether they are retained or evicted by Belady’s algorithm.

For each cache block involved in an eviction decision, a set of lightweight features is extracted solely from past access history and cache state, including recency and access frequency. Importantly, feature extraction does not rely on any future information provided by the oracle. These features, together with the Belady-derived labels, form a supervised training dataset. A lightweight decision tree classifier is trained on this dataset to learn correlations between historical access behavior and future reuse likelihood.

C. Online Cache Replacement Phase

During program execution, the cache operates using a standard set-associative organization. On a cache miss where the target set is full, the replacement policy must select a victim block for eviction. For each candidate block in the set, the same features used during training are computed online using only past access history and provided as input to the trained machine learning model. The model outputs a score representing the estimated cache-friendliness of each candidate block.

D. Uncertainty-Aware Eviction Decision

Rather than blindly trusting the machine learning predictions, the proposed policy evaluates the confidence of the

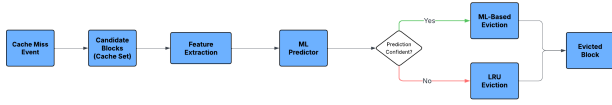


Fig. 2. Online runtime phase of the proposed uncertainty-aware cache replacement policy.

model’s output before making an eviction decision. Confidence is quantified based on the relative spread of prediction scores across candidate cache blocks within the target set.

Prediction confidence is quantified using the maximum predicted probability produced by the classifier across all candidate cache blocks in the target set. Let p_i denote the predicted cache-friendliness probability for candidate block i . The confidence metric is defined as:

$$\text{Confidence} = \max_i(p_i) \quad (1)$$

If the confidence exceeds a predefined threshold τ , the prediction is considered reliable, and the block with the lowest predicted cache-friendliness probability is selected for eviction. If the confidence is below τ , the replacement decision falls back to the Least Recently Used (LRU) policy. In our implementation, the confidence threshold is set to $\tau = 0.88$.

This threshold value was selected based on the empirical accuracy of the trained classifier during validation, which reached approximately 91%. Choosing a threshold slightly below the model’s accuracy provides a safety margin that reduces reliance on uncertain predictions while still enabling effective learning-based eviction decisions. This design ensures a balanced trade-off between performance improvement and robustness, allowing the policy to safely fall back to LRU when prediction confidence is insufficient.

V. IMPLEMENTATION AND EXPERIMENTAL SETUP

A. Cache Simulator

The proposed replacement policy is implemented using a trace-driven cache simulator developed in Python. The simulator models a set-associative cache and supports multiple replacement policies, including Least Recently Used (LRU) and the proposed hybrid machine learning-based policy.

For each memory access in the trace, the simulator updates cache state, records hit and miss events, and invokes the appropriate replacement policy upon a cache miss when the target set is full. All evaluated replacement policies operate under identical cache configurations to ensure a fair comparison.

The simulator reports total accesses, cache hits, cache misses, and hit rate for each workload.

B. Cache Configuration

All experiments are conducted using a fixed cache configuration to ensure fair comparison between replacement policies. The simulated cache is a 32 KB set-associative cache with the following parameters:

- Number of sets: 64
- Associativity: 8-way
- Block size: 64 bytes

This configuration results in a total cache capacity of $64 \times 8 \times 64 = 32,768$ bytes (32 KB).

C. Machine Learning Model Implementation

The learning-based component of the replacement policy is implemented using a single lightweight decision tree classifier. A decision tree is selected due to its low inference cost, interpretability, and suitability for hardware-oriented implementation. The model is implemented using the `scikit-learn` library.

The classifier operates on two input features only: recency, representing the number of accesses since the cache block was last referenced, and access frequency, representing the number of times the block has been accessed. No additional features, such as block age or future reuse information, are used.

To control model complexity and reduce the risk of overfitting, the decision tree is explicitly constrained using the following hyperparameters:

- Maximum tree depth: 8
- Minimum samples per split: 20
- Minimum samples per leaf: 10
- Class weighting: Balanced

With these constraints, the trained decision tree results in 241 leaf nodes, providing a compact model with low inference overhead.

These constraints ensure that the model learns general access patterns rather than memorizing specific traces.

D. Training Dataset Generation

Training data is generated offline using synthetic memory access traces designed to cover a wide range of cache behaviors. Ten distinct scenarios are considered, each producing a trace consisting of one million memory accesses.

Only the first 200,000 accesses from each trace are used to generate training data. Belady’s optimal replacement algorithm is simulated on this training portion to determine the optimal eviction decision at each cache miss. For every eviction event, candidate cache blocks are labeled according to Belady’s decision, producing supervised training samples.

Across all scenarios, this process yields approximately 13 million labeled samples. The dataset is split into 80% training and 20% testing subsets for model validation. The remaining 800,000 accesses per trace are reserved exclusively for performance evaluation and are not used during training, preventing memorization of access sequences.

Only features derived from past access history are used during training, ensuring that the model does not rely on future information provided by the oracle.

E. Online Replacement Policy Integration

During evaluation, the trained model is loaded and integrated into the cache simulator. On a cache miss where the target set is full, the same features used during training are

computed online for each candidate cache block using only historical access information.

The model outputs a cache-friendliness score for each candidate block. These scores are used to guide the eviction decision subject to the uncertainty-aware control mechanism described in Section IV.

F. Uncertainty-Aware Decision Control

To prevent harmful eviction decisions caused by unreliable predictions, an uncertainty-aware confidence mechanism is employed. Confidence is computed based on the spread of the predicted cache-friendliness scores among candidate blocks in the target set.

If the difference between the highest and lowest scores exceeds a predefined threshold, the model is considered confident, and the block with the lowest predicted cache-friendliness is selected for eviction. If confidence is insufficient, the policy falls back to the LRU replacement strategy.

This design guarantees LRU-equivalent behavior in worst-case scenarios while allowing machine learning to improve performance when predictions are reliable.

G. Evaluation Methodology

Performance evaluation is conducted using trace-driven simulation on the full memory access traces, each consisting of one million accesses per scenario. These evaluation traces extend well beyond the portions used during training, enabling long-term cache behavior to be observed and reducing the risk of overfitting.

Ten workloads are evaluated, covering mixed, hostile, scan-heavy, phase-based, and noisy access patterns. For each workload, the cache hit rate achieved by the proposed hybrid policy is compared against the LRU baseline under identical cache configurations.

In addition to hit rate, the proportion of eviction decisions handled by the machine learning model versus LRU fallback is recorded to analyze the behavior of the uncertainty-aware mechanism.

All experiments are conducted using the same simulator and configuration to ensure reproducibility and consistency.

VI. EXPERIMENTAL RESULTS AND DISCUSSION

A. Evaluation Metrics

Cache performance is evaluated using the cache hit rate, defined as the ratio of cache hits to total memory accesses. Hit rate is a widely used metric in cache replacement studies, as it directly reflects the effectiveness of eviction decisions.

In addition, the proportion of eviction decisions handled by the machine learning model versus the LRU fallback is recorded. This metric provides insight into the behavior of the uncertainty-aware confidence mechanism and its impact on performance.

TABLE I
CACHE HIT RATE COMPARISON ACROSS WORKLOADS

Workload	LRU	Hybrid	Improvement
Mixed	33.92%	42.37%	+8.45%
Hostile	0.93%	13.94%	+13.01%
Friendly	99.99%	99.99%	+0.00%
Scan Heavy	0.00%	16.67%	+16.67%
Phases	77.92%	77.58%	-0.35%
Alternating	47.50%	67.94%	+20.44%
Mixed Difficulty	49.98%	52.69%	+2.72%
Gradual Shift	59.80%	66.66%	+6.86%
Random Bursts	42.50%	49.75%	+7.25%
Noisy Patterns	74.01%	74.09%	+0.08%
Average	48.66%	56.17%	+7.51%

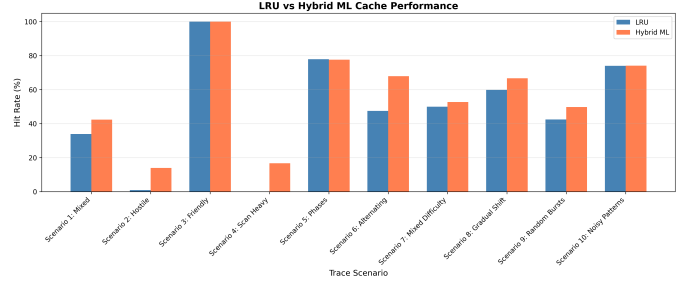


Fig. 3. Comparison of cache hit rate between LRU and the proposed hybrid machine learning replacement policy across all evaluated workloads.

B. Overall Performance Comparison

Table I summarizes the cache hit rates achieved by the LRU baseline and the proposed hybrid policy across all evaluated workloads. Each workload consists of one million memory accesses and is evaluated under identical cache configurations.

Across the ten evaluated workloads, the proposed hybrid policy improves the average cache hit rate by 7.51% compared to LRU. Performance gains are observed in nine out of ten scenarios, demonstrating consistent improvement across diverse access patterns.

C. Visualization of Results

Figure 3 provides a visual comparison of cache hit rates achieved by LRU and the proposed hybrid policy across all workloads. The figure highlights the relative performance differences and reinforces the trends observed in Table I.

As shown in Fig. 3, the hybrid policy significantly outperforms LRU in workloads where traditional heuristics fail, while matching or closely tracking LRU behavior in workloads with strong locality.

D. Workload-Specific Analysis

1) *LRU-Adversarial Workloads*: The largest improvements are observed in workloads designed to be adversarial to LRU, including the hostile, scan-heavy, and alternating scenarios. In these workloads, LRU frequently evicts cache blocks that are reused shortly thereafter, leading to excessive cache misses.

The proposed hybrid policy improves hit rate by 13.01% in the hostile workload and by 16.67% in the scan-heavy workload. The alternating workload exhibits the strongest gain, with

TABLE II
MACHINE LEARNING USAGE ACROSS WORKLOADS

Workload	ML Usage (%)
Mixed	95
Hostile	99
Friendly	0
Scan Heavy	98
Phases	75
Alternating	99
Mixed Difficulty	96
Gradual Shift	92
Random Bursts	85
Noisy Patterns	88

a 20.44% absolute improvement. These results demonstrate the effectiveness of learning reuse behavior beyond simple recency heuristics.

2) *LRU-Friendly and Stable Workloads*: In the friendly workload, which exhibits strong temporal locality, both LRU and the hybrid policy achieve a hit rate of 99.99%. In this case, the uncertainty-aware mechanism disables machine learning entirely, falling back to LRU for all eviction decisions. This confirms that the proposed policy does not interfere with workloads where LRU is already optimal.

Similarly, the noisy patterns workload shows only a marginal improvement of 0.08%, indicating conservative behavior when the benefit of learning-based decisions is limited.

3) *Phase-Based Workloads*: The phase-based workload is the only scenario in which a minor performance degradation is observed. The hybrid policy achieves a hit rate of 77.58%, compared to 77.92% for LRU, corresponding to a decrease of 0.35%.

Frequent working-set transitions reduce prediction reliability in this workload, causing the machine learning model to remain active for most eviction decisions. Importantly, the uncertainty-aware fallback mechanism bounds the magnitude of degradation, maintaining near-LRU performance and preventing catastrophic failures.

E. Machine Learning Usage Analysis

To further analyze the behavior of the uncertainty-aware mechanism, Table II reports the percentage of eviction decisions handled by the machine learning model for each workload.

In LRU-friendly workloads, such as the *Friendly* scenario, machine learning is completely disabled, resulting in behavior identical to the LRU baseline. This confirms that the uncertainty-aware mechanism correctly suppresses learning-based decisions when they provide no benefit, ensuring safe fallback behavior.

In contrast, LRU-adversarial workloads, including *Hostile*, *Scan Heavy*, and *Alternating*, exhibit very high machine learning usage, indicating that the model frequently produces confident predictions. In these cases, the learning-based policy is able to guide eviction decisions effectively, leading to substantial performance improvements over LRU.

Intermediate behavior is observed in workloads with higher variability, such as the *Phases*, *Random Bursts*, and *Noisy*

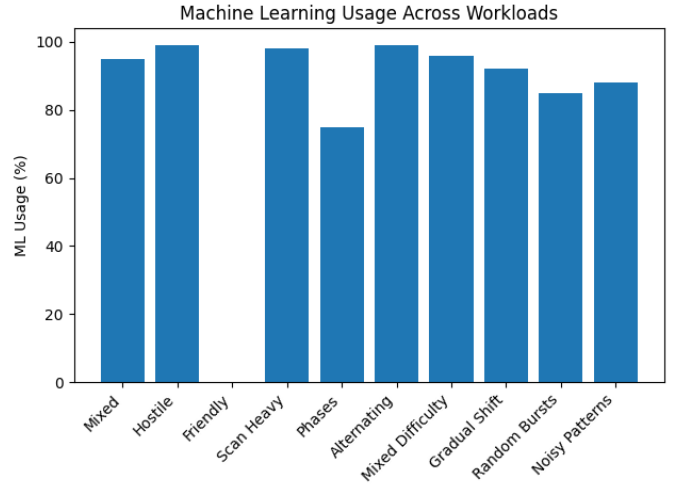


Fig. 4. Machine learning usage across workloads.

Patterns scenarios. In particular, the phase-based workload exhibits reduced ML usage (75%), reflecting increased prediction uncertainty caused by frequent working-set transitions. This selective reduction in ML usage limits the impact of mispredictions and explains the small, bounded performance degradation observed in this scenario.

Overall, these results demonstrate that the uncertainty-aware mechanism dynamically adjusts the reliance on machine learning based on workload characteristics. Machine learning is aggressively applied when predictions are reliable and conservatively reduced when uncertainty is high, ensuring robustness while still achieving performance gains.

F. Discussion

Overall, the experimental results demonstrate that the proposed Belady-guided hybrid replacement policy successfully combines the adaptiveness of machine learning with the robustness of traditional heuristics. The policy delivers substantial performance improvements in workloads where LRU fails, while preserving LRU-equivalent behavior in favorable or uncertain conditions.

The inclusion of prediction confidence ensures that machine learning augments cache replacement decisions without compromising worst-case performance, making the proposed approach suitable for practical deployment.

VII. CONCLUSION AND FUTURE WORK

This paper proposed a Belady-guided cache replacement policy that integrates lightweight supervised machine learning with an uncertainty-aware decision mechanism. By leveraging Belady's optimal algorithm as an offline oracle, the proposed approach learns to approximate optimal eviction decisions using only past access information available at runtime. To ensure robustness, machine learning predictions are applied selectively and safely fall back to LRU when confidence is low.

Experimental results obtained using a trace-driven cache simulator show that the proposed hybrid policy improves average cache hit rate by 7.51% over LRU across a range of synthetic workloads. Significant gains are observed in LRU-adversarial scenarios, while LRU-equivalent behavior is preserved in LRU-friendly workloads. Although the evaluation demonstrates the effectiveness of the approach, the scope of tested workloads remains limited and does not fully capture the diversity of real-world applications.

Future Work

Future research directions include:

- Evaluating the proposed policy on larger and more diverse workloads, including real application traces, to further validate generalization.
- Extending the approach to multi-level and multicore cache hierarchies.
- Exploring additional runtime features and alternative confidence estimation mechanisms to improve robustness under highly dynamic workloads.
- Investigating hardware-oriented implementations and cycle-accurate simulations to assess practical deployment overhead.

VIII. REFERENCES

REFERENCES

- [1] L. A. Belady, "A Study of Replacement Algorithms for a Virtual-Storage Computer," *IBM Systems Journal*, vol. 5, no. 2, pp. 78–101, 1966.
- [2] J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed. Morgan Kaufmann, 2019.
- [3] L. Souza and H. Freitas, "Reinforcement Learning-Based Cache Replacement Policies for Multicore Processors," 2023. [Online]. Available: <https://www.researchgate.net/publication/381182523>
- [4] A. Jain and C. Lin, "Glider: Applying Deep Learning to the Cache Replacement Problem," in *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2019.
- [5] Y. Liu et al., "PARROT: An Imitation Learning Approach for Cache Replacement," in *Proceedings of the 37th International Conference on Machine Learning (ICML)*, 2020.
- [6] A. Jain and C. Lin, "Hawkeye: Leveraging Belady's Algorithm for Improved Cache Replacement," in *Proceedings of the 43rd International Symposium on Computer Architecture (ISCA)*, 2016.
- [7] X. Lu et al., "CHROME: Concurrency-Aware Holistic Cache Management Using Online Reinforcement Learning," in *Proceedings of the IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2024.
- [8] G. Vietri et al., "LeCaR: Learning Cache Replacement via Online Regret Minimization," in *Proceedings of the USENIX Workshop on Hot Topics in Storage and File Systems (HotStorage)*, 2018.
- [9] M. K. Qureshi et al., "A Case for MLP-Aware Cache Replacement," in *Proceedings of the International Symposium on Computer Architecture (ISCA)*, 2006.
- [10] C. Wu et al., "PACMan: Prefetch-Aware Cache Management," in *Proceedings of the 44th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2011.
- [11] P. Pratheeksha and K. Revathi, "A Review of Machine-Learning Based Cache Replacement Policies," *International Journal of Engineering and Advanced Technology*, 2021.
- [12] Y. Wu and Y. Xie, "A Survey of Machine Learning for Computer Architecture and Systems," *arXiv preprint arXiv:2102.07952*, 2021.